

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN DỮ LIỆU LỚN

SỬ DỤNG SPARK ĐỂ DỰ ĐOÁN CHẤT LƯỢNG RƯỢU

Giảng viên hướng dẫn:	Ths. Phạm Đình Tài
Sinh viên thực hiện:	Hoàng Nguyên Hoàng
MSSV:	2311553873
Khoá:	2023-2027
Chuyên ngành:	KHOA HỌC DỮ LIỆU

Tp. HCM, tháng 08 năm 2026

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN DỮ LIỆU LỚN

SỬ DỤNG SPARK ĐỂ DỰ ĐOÁN CHẤT LƯỢNG RƯỢU

Giảng viên hướng dẫn:	Ths. Phạm Đình Tài
Sinh viên thực hiện:	Hoàng Nguyên Hoàng
MSSV:	2311553873
Khoá:	2023-2027
Chuyên ngành:	KHOA HỌC DỮ LIỆU

Tp. HCM, tháng 08 năm 2026

NHIỆM VỤ TIỂU LUẬN

Họ và tên: **HOÀNG NGUYỄN HOÀNG**

MSSV: **2311553873**

Chuyên ngành: **Khoa học dữ liệu**

Lớp: **23DTH2A**

Email: 2311553873@nttu.edu.vn

SĐT: **0365186686**

Tên đề tài: Sử dụng Spark dự đoán chất lượng rượu

Giáo viên hướng dẫn: **ThS. Phạm Đình Tài**

Thời gian thực hiện: **11/06/2026 đến 25/08/2026**

MÔ TẢ ĐỀ TÀI:

Đề tài ứng dụng Apache Spark và Spark MLlib vào bài toán dự đoán chất lượng rượu dựa trên các thuộc tính hóa học của bộ dữ liệu Wine Quality. Đề tài tập trung khảo sát, tiền xử lý và phân tích dữ liệu, từ đó xây dựng các mô hình học máy nhằm dự đoán chất lượng rượu. Qua đó, đề tài tìm hiểu khả năng kết hợp Apache Spark với Machine Learning trong quá trình xử lý, phân tích dữ liệu và xây dựng mô hình dự đoán. Kết quả thực hiện được sử dụng để đánh giá khả năng áp dụng các mô hình học máy vào bài toán phân loại chất lượng rượu, đồng thời làm cơ sở để so sánh hiệu quả giữa các mô hình được lựa chọn và đánh giá kết quả dự đoán thu được.

NỘI DUNG VÀ PHƯƠNG PHÁP:

Đề tài sử dụng Apache Spark và PySpark để đọc, kiểm tra, làm sạch và phân tích dữ liệu. Dữ liệu được khảo sát về cấu trúc, các thuộc tính, kiểu dữ liệu và chất lượng dữ liệu trước khi thực hiện các bước tiền xử lý cần thiết. Quá trình xử lý nhằm chuẩn bị dữ liệu phù hợp cho việc xây dựng và huấn luyện mô hình. VectorAssembler và StandardScaler được sử dụng để chuẩn bị và chuẩn hóa dữ liệu đầu vào phục vụ quá trình xây dựng mô hình.

Ba mô hình Logistic Regression, Decision Tree và Random Forest được xây dựng và huấn luyện bằng Spark MLlib. Các mô hình được đánh giá thông qua Accuracy, Precision, Recall, F1-score và Confusion Matrix nhằm đánh giá khả năng phân loại chất lượng rượu. Bên cạnh đó, đề tài thực hiện Feature Importance để phân tích mức độ quan trọng của các thuộc tính, sử dụng Cross Validation để đánh giá độ ổn định của mô hình và hỗ trợ lựa

chọn cấu hình phù hợp. Cuối cùng, mô hình được sử dụng để thực hiện dự đoán chất lượng cho một mẫu rượu mới và phân tích kết quả thu được nhằm đánh giá khả năng ứng dụng cũng như hiệu quả của mô hình trong bài toán.

YÊU CẦU:

Có kiến thức cơ bản về Big Data, Machine Learning, Python và Apache Spark; nắm được các khái niệm cơ bản liên quan đến dữ liệu lớn và học máy, đồng thời hiểu được quy trình tiền xử lý dữ liệu, xây dựng, huấn luyện và đánh giá mô hình học máy.

Có khả năng sử dụng PySpark và Spark MLlib để xử lý dữ liệu và xây dựng các mô hình Logistic Regression, Decision Tree và Random Forest. Biết sử dụng các chỉ số đánh giá để phân tích, so sánh kết quả giữa các mô hình; có khả năng nhận xét kết quả thực nghiệm, phân tích các thuộc tính ảnh hưởng đến kết quả dự đoán và thực hiện dự đoán trên dữ liệu mới.

Có khả năng tổng hợp và trình bày kết quả nghiên cứu thông qua bảng biểu, hình ảnh và các nhận xét phù hợp; đảm bảo nội dung báo cáo thể hiện được quá trình thực hiện từ khảo sát dữ liệu, tiền xử lý, xây dựng mô hình đến đánh giá và kết luận theo đúng quy định của tiểu luận. Đồng thời, báo cáo cần được trình bày rõ ràng, thống nhất, khoa học và đúng yêu cầu về hình thức của Bộ môn.

Nội dung và yêu cầu đã được thông qua Bộ môn.

TP.HCM, ngày 25 tháng 08 năm 2026

Q. TRƯỞNG BỘ MÔN

(Ký và ghi rõ họ tên)

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Trong quá trình thực hiện tiểu luận với đề tài “Sử dụng Apache Spark dự đoán chất lượng rượu”, nhóm đã có cơ hội vận dụng những kiến thức được học trong môn học về dữ liệu lớn, điện toán phân tán và học máy vào một bài toán thực tế.

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến giảng viên đã hướng dẫn, cung cấp những kiến thức nền tảng và định hướng cho nhóm trong quá trình thực hiện đề tài. Những kiến thức về Apache Spark, xử lý dữ liệu phân tán, phân tích dữ liệu và Machine Learning đã giúp nhóm có cơ sở để xây dựng quy trình thực nghiệm cho đề tài.

Nhóm cũng xin cảm ơn các thành viên trong nhóm đã cùng nhau trao đổi, phân chia công việc, kiểm tra kết quả và hoàn thiện nội dung tiểu luận. Trong quá trình thực hiện, nhóm đã gặp không ít khó khăn trong việc xử lý dữ liệu, xây dựng mô hình và đảm bảo các bước xử lý được thực hiện thống nhất trên môi trường Spark.

Mặc dù đã cố gắng hoàn thiện tiểu luận một cách tốt nhất, do giới hạn về thời gian, kiến thức và kinh nghiệm thực tế, tiểu luận chắc chắn vẫn còn những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ giảng viên để có thể tiếp tục hoàn thiện kiến thức và cải thiện đề tài trong tương lai.

Nhóm xin chân thành cảm ơn!

LỜI MỞ ĐẦU

Trong thế giới rượu vang, vang trắng là dòng sản phẩm phổ biến và được ưa chuộng bậc nhất nhờ hương vị tươi mát, thanh thoát cùng tính linh hoạt trong phối hợp ẩm thực. Chất lượng cảm quan của rượu phụ thuộc vào sự tương tác phức tạp giữa nhiều chỉ số hóa lý như độ pH, nồng độ cồn, độ axit, đường tồn dư và quy trình sản xuất. Tuy nhiên, việc đánh giá chất lượng truyền thống vốn phụ thuộc vào thử nghiệm cảm quan của các chuyên gia (sommelier) – một phương pháp vừa mang tính cảm tính, tốn kém chi phí, vừa khó mở rộng trên quy mô thương mại lớn.

Sự bùng nổ của Kỷ nguyên Dữ liệu lớn (Big Data) đã mở ra hướng đi đột phá cho ngành công nghệ thực phẩm và phân tích dữ liệu. Việc thu thập, lưu trữ và xử lý hàng loạt tập dữ liệu chỉ số hóa lý thực nghiệm cho phép chúng ta xây dựng các mô hình tự động hóa dự đoán chất lượng rượu một cách khách quan, chính xác và minh bạch. Dù vậy, thách thức đặt ra là làm sao xử lý và phân tích khối lượng dữ liệu khổng lồ một cách tối ưu, linh hoạt và đáp ứng tính thực thời.

Nhằm giải quyết bài toán này, nhóm đã lựa chọn ứng dụng Apache Spark – nền tảng tính toán phân tán hiệu năng cao – kết hợp với thư viện học máy Spark MLlib để xử lý song song dữ liệu và huấn luyện các mô hình dự đoán. Việc kết hợp giữa khoa học dữ liệu và công nghệ Big Data không chỉ giúp tối ưu hóa quy trình kiểm soát chất lượng đầu ra cho nhà sản xuất mà còn hỗ trợ doanh nghiệp và người tiêu dùng định giá sản phẩm một cách chuẩn xác.

Chính vì lý do đó, nhóm chúng em quyết định thực hiện đề tài "Dự đoán chất lượng rượu vang dựa trên nền tảng Apache Spark" trong khuôn khổ môn học Dữ liệu lớn.

Lời đầu tiên, nhóm xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Phạm Đình Tài, giảng viên hướng dẫn môn học. Nhờ có sự giảng dạy tận tình, những định hướng chuyên môn quý báu cùng sự đồng hành của Thầy trong suốt quá trình học tập và nghiên cứu, nhóm mới có thể hoàn thành đề tài này một cách trọn vẹn.

Dù đã cố gắng đầu tư và hoàn thiện đề tài với tinh thần nghiêm túc nhất, song bài báo cáo khó tránh khỏi những hạn chế thiếu sót. Nhóm kính mong nhận được những góp ý và đánh giá quý báu từ Thầy để bài nghiên cứu được hoàn thiện hơn.

Nhóm xin chân thành cảm ơn!

PHIẾU CHẤM THI TIỂU LUẬN/ĐỒ ÁN

Môn thi: Dữ Liệu Lớn Lớp học phần: 23DTH2B

Nhóm sinh viên thực hiện :

1. Nguyễn Minh Anh Tham gia đóng góp: 100%
2. Hoàng Nguyên Hoàng Tham gia đóng góp: 100%
3. Tham gia đóng góp:
4. Tham gia đóng góp:
5. Tham gia đóng góp:
6. Tham gia đóng góp:
7. Tham gia đóng góp:
8. Tham gia đóng góp:

Ngày thi: 25/08/2026 Phòng thi: Trục Tuyển 02

Đề tài tiểu luận/báo cáo của sinh viên : Sử dụng Spark dự đoán chất lượng rượu

Phản đánh giá của giảng viên (căn cứ trên thang rubrics của môn học):

Tiêu chí (theo CDR HP)	Đánh giá của GV	Điểm tối đa	Điểm đạt được
Cấu trúc của báo cáo			
Nội dung			
- Các nội dung thành phần			
- Lập luận			
- Kết luận			
Trình bày			
TỔNG ĐIỂM			

Giảng viên chấm thi

(ký, ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN	1
LỜI MỞ ĐẦU	2
DANH MỤC BẢNG	7
DANH MỤC HÌNH ẢNH	8
BẢNG KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT	9
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	10
1.1. Lý do chọn đề tài	10
1.2. Mục tiêu nghiên cứu	10
1.2.1. Mục tiêu cụ thể.....	10
1.3. Đối tượng và phạm vi nghiên cứu	11
1.3.1. Đối tượng nghiên cứu	11
1.3.2. Phạm vi nghiên cứu	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	13
2.1. Tổng quan về Big Data	13
2.1.1. Khái niệm Big Data	13
2.1.2. Đặc trưng của Big Data	13
2.2. Tổng quan về Apache Spark	14
2.2.1. Khái niệm Apache Spark	14
2.2.2. Kiến trúc Apache Spark.....	14
2.2.3. Các thành phần chính của Apache Spark	14
2.3. Spark MLlib	15
2.3.1. Khái niệm MLlib	15
2.3.2. Các nhóm thuật toán trong MLlib.....	15
2.4. Các thuật toán sử dụng	16
2.4.1. Logistic Regression	16
2.4.2. Decision Tree	16
2.4.3. Random Forest	16
2.5. VectorAssembler	17
2.6. StandardScaler	17
2.7. Các chỉ số đánh giá mô hình	18
2.7.1. Accuracy	18
2.7.2. Precision	18
2.7.3. Recall	18
2.7.4. F1-score	18
CHƯƠNG 3: QUY TRÌNH XỬ LÝ VÀ XÂY DỰNG MÔ HÌNH	19

3.1. Cài đặt và môi trường thực nghiệm	19
3.1.1. Google Colab	19
3.1.2. Cài đặt PySpark	19
3.1.3. Khởi tạo SparkSession	19
3.1.4. Kiểm tra phiên bản Spark	19
3.2. Phân tích và tiền xử lý dữ liệu	20
3.2.1. Giới thiệu bài toán	20
3.2.2. Bộ dữ liệu Wine Quality	20
3.2.3. Đọc dữ liệu	21
3.2.4. Hiển thị dữ liệu	22
3.2.5. Kiểm tra Schema	22
3.2.6. Kiểm tra kích thước dữ liệu	22
3.2.7. Thống kê mô tả	23
3.2.8. Kiểm tra giá trị thiếu	23
3.2.9. Xử lý giá trị thiếu	23
3.2.10. Kiểm tra dữ liệu bất hợp lệ	24
3.2.11. Xử lý dữ liệu bất hợp lệ	24
3.2.12. Phát hiện Outlier bằng IQR	25
3.2.13. Xử lý biến Quality	25
3.2.14. Phân tích khám phá dữ liệu	26
3.2.15. Phân tích tương quan	29
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	32
4.1. Chuẩn bị đặc trưng	32
4.2. Vector hóa dữ liệu	32
4.3. Chuẩn hóa	32
4.4. Chia dữ liệu Train/Test	32
4.5. Mô hình Logistic Regression	33
4.6. Mô hình Decision Tree	33
4.7. Mô hình Random Forest	34
4.8. Hàm đánh giá mô hình	34
4.9. So sánh các mô hình	35
4.10. Biểu đồ so sánh	35
4.11. Confusion Matrix	36
4.12. Classification Report	37
4.13. Feature Importance	37
4.14. Cross Validation	38

4.15. Dự đoán mẫu rượu mới	38
CHƯƠNG 5: KẾT LUẬN	39
5.1. Kết luận	39
5.2. Ưu điểm của đề tài	39
5.3. Hạn chế của đề tài	40
5.4. Hướng phát triển.....	40
TÀI LIỆU THAM KHẢO	41
PHỤ LỤC	42

DANH MỤC BẢNG

Bảng 3.1 – Bảng các thuộc tính của bộ dữ liệu Wine Quality.....	21
---	----

DANH MỤC HÌNH ẢNH

Hình 3.1. Dữ liệu Wine Quality sau khi đọc bằng Spark	22
Hình 3.2. Schema của bộ dữ liệu.....	22
Hình 3.3: Phân bố chất lượng rượu	27
Hình 3.4: Phân bố Alcohol.....	27
Hình 3.5: Phân bố Volatile Acidity	28
Hình 3.6: Phân bố Sulphates	28
Hình 3.7: Phân bố Citric Acid.....	29
Hình 3.8: Phân bố Density	29
Hình 3.9: Ma trận tương quan giữa các thuộc tính.....	30
Hình 3.10: Mức độ tương quan giữa các thuộc tính và Quality	31
Hình 4.1: So sánh hiệu quả các mô hình	35
Hình 4.2: Confusion Matrix – Random Forest	36
Hình 4.4: Kết quả dự đoán mẫu rượu mới	38

BẢNG KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Ký hiệu	Cụm từ viết đầy đủ	Nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CSV	Comma-Separated Values	Tập dữ liệu dạng bảng phân tách bằng dấu phẩy
EDA	Exploratory Data Analysis	Phân tích khám phá dữ liệu
IQR	Interquartile Range	Khoảng tứ phân vị
ML	Machine Learning	Học máy
MLlib	Machine Learning Library	Thư viện học máy của Apache Spark
PySpark	Python API for Apache Spark	Giao diện Apache Spark bằng Python
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, sự phát triển của công nghệ thông tin làm cho lượng dữ liệu được tạo ra ngày càng lớn. Việc xử lý và khai thác hiệu quả những dữ liệu này trở thành một vấn đề quan trọng trong nhiều lĩnh vực. Apache Spark là một nền tảng xử lý dữ liệu phân tán mạnh mẽ, hỗ trợ xử lý dữ liệu lớn và cung cấp thư viện Spark MLlib phục vụ cho các bài toán Machine Learning.

Trong lĩnh vực sản xuất rượu, chất lượng sản phẩm chịu ảnh hưởng bởi nhiều yếu tố hóa học như độ acid, hàm lượng đường, chlorides, sulfur dioxide, density, pH, sulphates và nồng độ alcohol. Các thông số này có mối quan hệ nhất định với chất lượng rượu. Vì vậy, việc sử dụng dữ liệu hóa học để xây dựng mô hình dự đoán chất lượng là một bài toán phù hợp để áp dụng các kỹ thuật phân tích dữ liệu và học máy.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài “Sử dụng Apache Spark dự đoán chất lượng rượu” nhằm tìm hiểu khả năng ứng dụng Apache Spark và Spark MLlib vào một bài toán thực tế. Đề tài thực hiện các bước từ khảo sát, tiền xử lý và phân tích dữ liệu đến xây dựng mô hình dự đoán.

Trong quá trình thực hiện, nhóm sử dụng ba mô hình Logistic Regression, Decision Tree và Random Forest để dự đoán chất lượng rượu. Các mô hình được đánh giá thông qua những chỉ số như Accuracy, Precision, Recall và F1-score. Bên cạnh đó, nhóm sử dụng Feature Importance và Cross Validation để phân tích đặc trưng và đánh giá mô hình.

Thông qua đề tài, nhóm có thể củng cố kiến thức về Big Data, Apache Spark và Machine Learning, đồng thời hiểu rõ hơn quy trình xây dựng một mô hình dự đoán từ dữ liệu thực tế. Đây cũng là cơ sở để nhóm tiếp tục phát triển các bài toán phân tích và dự đoán trên những tập dữ liệu có quy mô lớn hơn trong tương lai.

1.2. Mục tiêu nghiên cứu

Xây dựng quy trình sử dụng Apache Spark và Spark MLlib để xử lý, phân tích dữ liệu và dự đoán chất lượng rượu dựa trên các đặc trưng hóa học của bộ dữ liệu Wine Quality.

1.2.1. Mục tiêu cụ thể

Tìm hiểu tổng quan về Big Data, Apache Spark và Spark MLlib, đặc biệt là khả năng ứng dụng Spark trong xử lý dữ liệu và Machine Learning.

Khảo sát bộ dữ liệu Wine Quality, tìm hiểu ý nghĩa các thuộc tính hóa học và biến Quality dùng để dự đoán chất lượng rượu.

Thực hiện kiểm tra và tiền xử lý dữ liệu, bao gồm kiểm tra cấu trúc, giá trị thiếu, dữ liệu bất hợp lệ và phát hiện ngoại lệ bằng phương pháp IQR.

Phân tích dữ liệu thông qua thống kê mô tả, biểu đồ phân bố và ma trận tương quan nhằm tìm hiểu đặc điểm của bộ dữ liệu.

Chuẩn bị dữ liệu cho mô hình bằng VectorAssembler và StandardScaler, sau đó chia dữ liệu thành tập huấn luyện và tập kiểm tra.

Xây dựng và huấn luyện ba mô hình Logistic Regression, Decision Tree và Random Forest bằng Spark MLlib để dự đoán chất lượng rượu.

Đánh giá và so sánh các mô hình thông qua Accuracy, Precision, Recall, F1-score và Confusion Matrix.

Phân tích Feature Importance, sử dụng Cross Validation để đánh giá cấu hình Random Forest và lựa chọn mô hình phù hợp.

Thực hiện dự đoán chất lượng cho một mẫu rượu mới, từ đó đánh giá khả năng ứng dụng của mô hình vào bài toán thực tế.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là quá trình ứng dụng Apache Spark và Spark MLlib vào bài toán phân loại chất lượng rượu dựa trên các thuộc tính hóa học.

Trong đó, dữ liệu đầu vào bao gồm 11 thuộc tính hóa học của rượu:

1. Fixed Acidity.
2. Volatile Acidity.
3. Citric Acid.
4. Residual Sugar.
5. Chlorides.
6. Free Sulfur Dioxide.
7. Total Sulfur Dioxide.
8. Density.
9. pH.

10. Sulphates.

11. Alcohol.

Biến mục tiêu của bài toán là quality.

1.3.2. Phạm vi nghiên cứu

Dưới đây là phiên bản được mở rộng chi tiết, bổ sung thêm các phân tích kỹ thuật và văn phong học thuật chuyên sâu hơn để đoạn văn đượm chất báo cáo đề án tốt nghiệp:

Đề tài này được nghiên cứu, thiết kế và triển khai trong môi trường tính toán đám mây hiện đại Google Colab, kết hợp đồng bộ giữa ngôn ngữ lập trình Python và thư viện mã nguồn mở PySpark. Toàn bộ chu trình xử lý thông tin — bao gồm việc tiếp nhận, làm sạch, biến đổi dữ liệu lớn cho đến các bước huấn luyện, tinh chỉnh và kiểm định mô hình — đều được vận hành xuyên suốt trên nền tảng kiến trúc phân tán mạnh mẽ của hệ sinh thái Apache Spark.

Về khía cạnh dữ liệu và bài toán, đề tài tập trung khai thác bộ dữ liệu dạng bảng kinh điển Wine Quality nhằm giải quyết bài toán phân loại nhiều lớp (multi-class classification). Cụ thể, các mức đánh giá chất lượng rượu (Quality) ban đầu bao gồm các nhãn từ 4 đến 8 (tương ứng với các mức 4, 5, 6, 7 và 8). Để tương thích tuyệt đối với các yêu cầu đầu vào khắt khe của các thuật toán phân loại trong thư viện học máy phân tán Spark MLlib, các giá trị nhãn này đã được tiến hành ánh xạ, chuẩn hóa và chuyển đổi sang dạng số nguyên tuần tự trong khoảng từ 0 đến 4. Quá trình này giúp mô hình tối ưu hóa khả năng học tập, đồng thời giảm thiểu sai số trong quá trình phân phối tác vụ trên các nút tính toán.

Về mặt ý nghĩa và phạm vi ứng dụng, đề tài này chủ yếu mang tính chất nghiên cứu khoa học, thử nghiệm công nghệ và học tập chuyên sâu nhằm minh họa một cách trực quan, sinh động quy trình ứng dụng các kỹ thuật xử lý dữ liệu lớn (Big Data) vào thực tế bài toán Machine Learning. Cần phải khẳng định rằng, các kết quả dự đoán, phân tích hay đánh giá tự động do hệ thống mô hình tạo ra chỉ mang giá trị định hướng học thuật và tham khảo kỹ thuật trong khuôn khổ đề án, tuyệt đối không được sử dụng để thay thế cho các tiêu chuẩn kiểm định chất lượng rượu chuyên nghiệp, thương mại hay các quy chuẩn kiểm định công nghiệp ngoài thực tế.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Big Data

2.1.1. Khái niệm Big Data

Big Data, hay dữ liệu lớn, là thuật ngữ dùng để mô tả những tập dữ liệu có kích thước, tốc độ phát sinh và mức độ đa dạng lớn đến mức các phương pháp xử lý dữ liệu truyền thống gặp khó khăn trong việc lưu trữ, quản lý và phân tích.

Sự phát triển của Internet, thiết bị di động, mạng xã hội, hệ thống cảm biến và các ứng dụng trực tuyến đã tạo ra lượng dữ liệu ngày càng lớn. Dữ liệu không chỉ tăng về số lượng mà còn xuất hiện dưới nhiều dạng khác nhau như dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc.

Trong bối cảnh đó, các hệ thống xử lý dữ liệu lớn cần có khả năng lưu trữ và xử lý dữ liệu trên nhiều máy tính thay vì phụ thuộc hoàn toàn vào một máy tính duy nhất. Đây là cơ sở cho sự phát triển của các nền tảng xử lý phân tán như Hadoop và Apache Spark.

2.1.2. Đặc trưng của Big Data

Big Data thường được mô tả thông qua các đặc trưng 5V.

- Volume

Volume thể hiện khối lượng dữ liệu. Khi số lượng dữ liệu tăng lên rất lớn, việc lưu trữ và xử lý trên một máy tính đơn lẻ có thể không còn hiệu quả.

- Velocity

Velocity thể hiện tốc độ dữ liệu được tạo ra, truyền tải và xử lý. Một số hệ thống cần khả năng xử lý dữ liệu gần thời gian thực.

- Variety

Variety thể hiện sự đa dạng về định dạng dữ liệu. Dữ liệu có thể tồn tại dưới dạng bảng, văn bản, hình ảnh, video, JSON, XML hoặc dữ liệu từ cảm biến.

- Veracity

Veracity thể hiện mức độ chính xác và đáng tin cậy của dữ liệu. Dữ liệu thực tế có thể chứa giá trị thiếu, sai lệch hoặc bất thường.

- Value

Value thể hiện giá trị mà tổ chức có thể khai thác từ dữ liệu. Việc xử lý dữ liệu chỉ thực sự có ý nghĩa khi kết quả có thể hỗ trợ phân tích, dự đoán hoặc ra quyết định.

2.2. Tổng quan về Apache Spark

2.2.1. Khái niệm Apache Spark

Apache Spark là một nền tảng xử lý dữ liệu phân tán đa ngôn ngữ, được sử dụng trong kỹ thuật dữ liệu, khoa học dữ liệu và Machine Learning. Spark được phát triển từ dự án nghiên cứu tại AMPLab thuộc Đại học California, Berkeley và sau đó được chuyển giao cho Apache Software Foundation để tiếp tục phát triển. Apache Spark hỗ trợ nhiều ngôn ngữ như Scala, Java và Python.

Một trong những điểm nổi bật của Apache Spark là khả năng thực hiện nhiều thao tác xử lý dữ liệu trên bộ nhớ, giúp hạn chế việc phải đọc và ghi dữ liệu trung gian liên tục. Điều này đặc biệt hữu ích đối với các bài toán cần thực hiện nhiều phép biến đổi dữ liệu hoặc huấn luyện mô hình Machine Learning.

Spark không yêu cầu phải sử dụng một hệ thống file riêng mà có thể làm việc với nhiều nguồn lưu trữ khác nhau như HDFS, Cassandra hoặc Amazon S3. Spark cũng hỗ trợ nhiều định dạng dữ liệu phổ biến như CSV và JSON.

2.2.2. Kiến trúc Apache Spark

Tổng quan kiến trúc: Kiến trúc Spark gồm các thành phần chính như Driver Program, Cluster Manager, Worker Node và Executor, giúp hệ thống có khả năng phân chia công việc thành nhiều tác vụ và thực hiện chúng song song trên nhiều tài nguyên tính toán khác nhau.

Driver Program: Là thành phần điều khiển chính của ứng dụng Spark, chịu trách nhiệm phân tích chương trình, tạo các tác vụ và điều phối quá trình thực thi một cách hiệu quả.

Cluster Manager chịu trách nhiệm quản lý tài nguyên của cụm máy tính. Spark có thể hoạt động với nhiều loại trình quản lý cụm khác nhau.

Worker Node là các máy thực hiện công việc xử lý dữ liệu.

Executor là tiến trình chạy trên Worker Node, chịu trách nhiệm thực hiện các Task được Driver phân phối.

Cách tổ chức này giúp Spark có khả năng phân chia công việc thành nhiều tác vụ và thực hiện chúng song song trên nhiều tài nguyên tính toán.

2.2.3. Các thành phần chính của Apache Spark

- Spark Core

Spark Core là nền tảng cơ bản của Spark, chịu trách nhiệm quản lý bộ nhớ, lập lịch, phân phối công việc, xử lý lỗi và tương tác với hệ thống lưu trữ. Spark Core cung cấp API cho nhiều ngôn ngữ như Java, Scala, Python và R.

- Spark SQL

Spark SQL hỗ trợ xử lý dữ liệu có cấu trúc thông qua DataFrame và SQL. Thành phần này phù hợp với những bài toán cần thao tác trên dữ liệu dạng bảng.

- Spark Streaming

Spark Streaming hỗ trợ xử lý dữ liệu dạng luồng. Thành phần này có thể tiếp nhận dữ liệu liên tục và thực hiện xử lý theo từng khoảng thời gian.

- MLlib

MLlib là thư viện Machine Learning của Spark. Thư viện này hỗ trợ nhiều nhóm thuật toán như phân loại, hồi quy, phân cụm và các phương pháp khai phá dữ liệu. MLlib tận dụng khả năng xử lý phân tán của Spark để phục vụ các bài toán học máy.

- GraphX

GraphX được sử dụng để xử lý dữ liệu dạng đồ thị, hỗ trợ các bài toán phân tích quan hệ giữa các thực thể.

2.3. Spark MLlib

2.3.1. Khái niệm MLlib

MLlib là thư viện Machine Learning được tích hợp trong Apache Spark, được thiết kế để hỗ trợ quá trình xây dựng, huấn luyện và đánh giá các mô hình học máy trên dữ liệu có quy mô lớn.

Điểm quan trọng của MLlib trong đề tài là nhóm có thể thực hiện quá trình Machine Learning trực tiếp trên DataFrame của Spark. Điều này giúp hạn chế việc phải chuyển dữ liệu sang một hệ thống xử lý khác trong quá trình xây dựng mô hình.

2.3.2. Các nhóm thuật toán trong MLlib

MLlib cung cấp nhiều nhóm thuật toán Machine Learning, trong đó có các bài toán:

- Classification.
- Regression.
- Clustering.

- Collaborative Filtering.
- Dimensionality Reduction.

Đối với đề tài này, nhóm tập trung vào Classification vì biến mục tiêu Quality được biểu diễn thành nhiều lớp tương ứng với các mức chất lượng rượu.

2.4. Các thuật toán sử dụng

2.4.1. Logistic Regression

Logistic Regression là thuật toán học máy thường được sử dụng cho các bài toán phân loại. Thay vì trực tiếp dự đoán một giá trị liên tục, mô hình xác định xác suất một mẫu thuộc về từng lớp.

Trong đề tài, Logistic Regression được sử dụng như một mô hình để so sánh với các thuật toán cây quyết định. Mô hình sử dụng vector đặc trưng đã được chuẩn hóa bởi StandardScaler.

Ưu điểm của Logistic Regression là cấu trúc tương đối đơn giản, thời gian huấn luyện thường không quá lớn và kết quả có thể được sử dụng làm mô hình cơ sở.

Tuy nhiên, mô hình có thể gặp hạn chế khi mối quan hệ giữa các đặc trưng và biến mục tiêu có tính phi tuyến hoặc phức tạp.

2.4.2. Decision Tree

Decision Tree là thuật toán phân loại dựa trên cấu trúc cây quyết định. Dữ liệu được phân chia thành các nhánh dựa trên các thuộc tính đầu vào. Quá trình phân chia tiếp tục cho đến khi đạt điều kiện dừng.

Một cây quyết định gồm các nút, nhánh và nút lá. Nút gốc đại diện cho điểm phân chia đầu tiên, các nút bên trong tiếp tục chia dữ liệu và nút lá biểu diễn kết quả cuối cùng.

Ưu điểm của Decision Tree là dễ giải thích và có khả năng mô hình hóa các quan hệ phi tuyến. Tuy nhiên, nếu cây quá sâu, mô hình có thể bị overfitting.

Trong Colab, nhóm giới hạn maxDepth=5 nhằm kiểm soát độ phức tạp của cây.

2.4.3. Random Forest

Random Forest là thuật toán ensemble xây dựng nhiều Decision Tree và kết hợp kết quả của chúng để đưa ra dự đoán cuối cùng.

Việc sử dụng nhiều cây giúp mô hình giảm sự phụ thuộc vào một cây quyết định duy nhất. Random Forest có khả năng mô hình hóa các quan hệ phức tạp và thường hoạt động tốt trên dữ liệu dạng bảng.

Trong đề tài, Random Forest được thiết lập với 100 cây và `maxDepth=8`. Ngoài khả năng dự đoán, Random Forest còn cung cấp Feature Importance, giúp nhóm phân tích mức độ quan trọng của 11 đặc trưng hóa học.

2.5. VectorAssembler

Machine Learning trong Spark yêu cầu các đặc trưng đầu vào được biểu diễn dưới dạng vector. Tuy nhiên, dữ liệu ban đầu gồm nhiều cột riêng biệt.

VectorAssembler được sử dụng để kết hợp các cột đặc trưng thành một cột vector duy nhất.

```
assembler = VectorAssembler(  
    inputCols=feature_cols,  
    outputCol="features"  
)
```

Sau bước này, các thuộc tính hóa học được kết hợp thành cột features, tạo đầu vào cho các mô hình Spark MLlib.

2.6. StandardScaler

Sự khác biệt về thang đo: Các thuộc tính hóa học trong dữ liệu thường có các thang đo và miền giá trị rất khác nhau. Ví dụ, nồng độ alcohol (thường dao động từ khoảng 8 đến 15) sẽ có khoảng giá trị hoàn toàn khác biệt so với các thành phần như sulfur dioxide (có thể lên tới vài chục hoặc hàng trăm) hay mật độ density (các giá trị nhỏ gần sát 1).

Vai trò của StandardScaler: Để giải quyết vấn đề này, lớp StandardScaler được sử dụng như một công cụ đắc lực trong việc chuẩn hóa các đặc trưng. Thuật toán này giúp đưa các biến về cùng một mặt bằng thang đo chung với trung bình = 0 và độ lệch chuẩn = 1.

Lợi ích đối với mô hình học máy: Việc chuẩn hóa dữ liệu này giúp các đặc trưng đóng góp công bằng hơn vào quá trình tính toán của mô hình, tránh việc các biến có biên độ số lớn lấn át các biến có biên độ nhỏ, từ đó nâng cao độ ổn định và hiệu suất tổng thể của thuật toán.

Trong Colab:

```

scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures",
    withStd=True,
    withMean=True
)

scaler_model = scaler.fit(df_vector)
df_scaled = scaler_model.transform(df_vector)

df_scaled.select("label", "features", "scaledFeatures").show(5, truncate=False)

```

	label	features	scaledFeatures
2		[7.0, 0.27, 0.36, 20.7, 0.045, 45.0, 170.0, 1.001, 3.0, 0.45, 8.8]	[0.17753670710983124, -0.07797368113039121, 0.21358365882404964, 2.8204
2		[6.3, 0.3, 0.34, 1.6, 0.049, 14.0, 132.0, 0.994, 3.3, 0.49, 9.5]	[-0.6591418757610269, 0.22106258674037366, 0.048410023229076726, -0.945
2		[8.1, 0.28, 0.4, 6.9, 0.05, 30.0, 97.0, 0.9951, 3.26, 0.44, 10.1]	[1.4923173373354648, 0.021705074826530595, 0.5439309300139964, 0.099244
2		[7.2, 0.23, 0.32, 8.5, 0.058, 47.0, 186.0, 0.9956, 3.19, 0.4, 9.9]	[0.41658773078721945, -0.4766887049580782, -0.11676361236589665, 0.4147
2		[7.2, 0.23, 0.32, 8.5, 0.058, 47.0, 186.0, 0.9956, 3.19, 0.4, 9.9]	[0.41658773078721945, -0.4766887049580782, -0.11676361236589665, 0.4147

only showing top 5 rows

Sau khi chuẩn hóa, dữ liệu được lưu trong cột scaledFeatures.

2.7. Các chỉ số đánh giá mô hình

2.7.1. Accuracy

Accuracy thể hiện tỷ lệ số mẫu được mô hình dự đoán chính xác trên tổng số mẫu.

$$Accuracy = \frac{Số dự đoán đúng}{Tổng số mẫu}$$

Accuracy càng cao cho thấy mô hình có tỷ lệ dự đoán đúng càng lớn.

2.7.2. Precision

Precision phản ánh mức độ chính xác của những dự đoán mà mô hình đưa ra cho một lớp.

$$Precision = \frac{TP}{TP + FP}$$

2.7.3. Recall

Recall thể hiện khả năng mô hình phát hiện đúng những mẫu thực sự thuộc một lớp.

$$Recall = \frac{TP}{TP + FN}$$

2.7.4. F1-score

F1-score là chỉ số kết hợp Precision và Recall.

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Trong đề tài, F1-score được sử dụng làm một trong những tiêu chí quan trọng khi so sánh mô hình.

CHƯƠNG 3: QUY TRÌNH XỬ LÝ VÀ XÂY DỰNG MÔ HÌNH

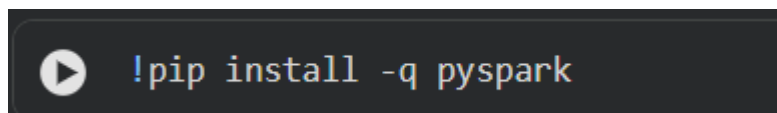
3.1. Cài đặt và môi trường thực nghiệm

3.1.1. Google Colab

Google Colab được sử dụng làm môi trường thực nghiệm vì cung cấp môi trường Python trực tuyến và có thể cài đặt PySpark trực tiếp. Cách tiếp cận này phù hợp với mục tiêu của đề tài vì nhóm không cần phải thực hiện toàn bộ quá trình cấu hình Spark thủ công trên máy cá nhân.

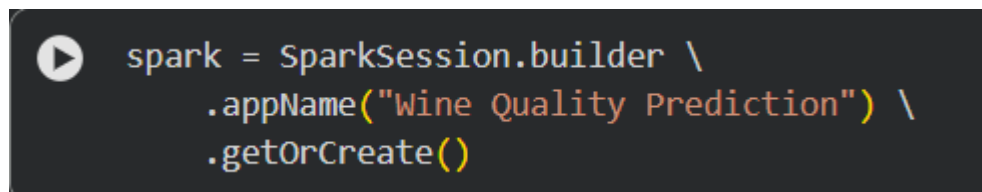
PySpark được cài đặt trực tiếp trong môi trường Google Colab để phục vụ quá trình thực nghiệm.

3.1.2. Cài đặt PySpark



```
!pip install -q pyspark
```

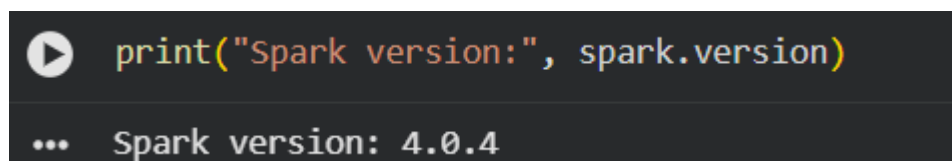
3.1.3. Khởi tạo SparkSession



```
spark = SparkSession.builder \
    .appName("Wine Quality Prediction") \
    .getOrCreate()
```

SparkSession là điểm khởi đầu để làm việc với Spark SQL và DataFrame. Trong đề tài, SparkSession được sử dụng để đọc dữ liệu, xử lý DataFrame và huấn luyện các mô hình Spark MLlib.

3.1.4. Kiểm tra phiên bản Spark



```
print("Spark version:", spark.version)
... Spark version: 4.0.4
```

3.2. Phân tích và tiền xử lý dữ liệu

3.2.1. Giới thiệu bài toán

Mục tiêu của đề tài: Bài toán cốt lõi của đề tài là xây dựng mô hình học máy tự động dự đoán và đánh giá chất lượng của một mẫu rượu dựa trên tập hợp các thông số hóa học đặc trưng, giúp hỗ trợ quá trình phân tích nhanh chóng và khách quan.

Xác định các biến trong mô hình:

Biến đầu vào (Features): Bao gồm toàn bộ các thông số hóa học quan trọng được trích xuất từ dữ liệu (như độ chua, nồng độ đường, pH, nồng độ alcohol, sulfur dioxide, v.v.).

Biến mục tiêu (Target/Label): Điểm đánh giá chất lượng (quality) đóng vai trò là nhãn cần dự đoán của hệ thống.

Chuyển đổi bài toán phân loại đa lớp: Do điểm số quality thực tế có nhiều mức giá trị phân tán khác nhau, nhóm nghiên cứu đã định hướng và quy hoạch bài toán thành một bài toán phân loại đa lớp (multi-class classification) nhằm phân định rõ ràng các cấp độ chất lượng rượu.

Quy trình ánh xạ nhãn dữ liệu: Trong môi trường thực thi trên Google Colab, dải giá trị quality từ 4 đến 8 được tiến hành chuẩn hóa, dời khoảng và chuyển đổi tuần tự thành hệ thống nhãn số nguyên từ 0 đến 4 để tương thích tuyệt đối với các yêu cầu đầu vào của các thuật toán phân loại trong Spark MLlib:

Trong Colab hiện tại, các mức Quality từ 4 đến 8 được sử dụng:

Quality 4 → Label 0

Quality 5 → Label 1

Quality 6 → Label 2

Quality 7 → Label 3

Quality 8 → Label 4

Cách chuyển đổi này được sử dụng thống nhất trong quá trình xây dựng mô hình của đề tài.

3.2.2. Bộ dữ liệu Wine Quality

Bộ dữ liệu gồm các thuộc tính hóa học của rượu và điểm chất lượng tương ứng.

STT	Thuộc tính	Ý nghĩa
1	fixed acidity	Độ acid cố định
2	volatile acidity	Độ acid dễ bay hơi
3	citric acid	Hàm lượng acid citric
4	residual sugar	Lượng đường dư
5	chlorides	Hàm lượng chloride
6	free sulfur dioxide	SO ₂ tự do
7	total sulfur dioxide	Tổng SO ₂
8	density	Khối lượng riêng
9	pH	Độ pH
10	sulphates	Hàm lượng sulphate
11	alcohol	Nồng độ cồn

Bảng 3.1 – Bảng các thuộc tính của bộ dữ liệu Wine Quality.

Biến mục tiêu:

quality – điểm chất lượng rượu

3.2.3. Đọc dữ liệu

```
df_spark = spark.read.csv(
    DATA_PATH,
    header=True,
    inferSchema=True
)
```

Trong đó header=True cho phép Spark sử dụng dòng đầu tiên của file làm tên cột. inferSchema=True giúp Spark tự suy luận kiểu dữ liệu của các thuộc tính.

Sau khi đọc dữ liệu, nhóm tiến hành kiểm tra dữ liệu để đảm bảo file được đọc chính xác.

3.2.4. Hiện thị dữ liệu

```
df_spark.show(10, truncate=False)
```

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.001	3.0	0.45
6.3	0.3	0.34	1.6	0.049	14.0	132.0	0.994	3.3	0.49
8.1	0.28	0.4	6.9	0.05	30.0	97.0	0.9951	3.26	0.44
7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.4
7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.4
8.1	0.28	0.4	6.9	0.05	30.0	97.0	0.9951	3.26	0.44
6.2	0.32	0.16	7.0	0.045	30.0	136.0	0.9949	3.18	0.47
7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.001	3.0	0.45
6.3	0.3	0.34	1.6	0.049	14.0	132.0	0.994	3.3	0.49
8.1	0.22	0.43	1.5	0.044	28.0	129.0	0.9938	3.22	0.45

only showing top 10 rows

Hình 3.1. Dữ liệu Wine Quality sau khi đọc bằng Spark

Kết quả cho phép quan sát trực tiếp những dòng đầu tiên của bộ dữ liệu, từ đó kiểm tra tên thuộc tính, giá trị và cấu trúc ban đầu của dữ liệu.

3.2.5. Kiểm tra Schema

```
df_spark.printSchema()
```

```
root
|-- fixed acidity: double (nullable = true)
|-- volatile acidity: double (nullable = true)
|-- citric acid: double (nullable = true)
|-- residual sugar: double (nullable = true)
|-- chlorides: double (nullable = true)
|-- free sulfur dioxide: double (nullable = true)
|-- total sulfur dioxide: double (nullable = true)
|-- density: double (nullable = true)
|-- pH: double (nullable = true)
|-- sulphates: double (nullable = true)
|-- alcohol: double (nullable = true)
|-- quality: integer (nullable = true)
```

Hình 3.2. Schema của bộ dữ liệu

Schema giúp xác định kiểu dữ liệu của từng thuộc tính. Đây là bước quan trọng vì các thuật toán Machine Learning yêu cầu dữ liệu đầu vào phải có kiểu dữ liệu phù hợp.

3.2.6. Kiểm tra kích thước dữ liệu

```
print("Số dòng:", df_spark.count())
print("Số cột:", len(df_spark.columns))
```

```
... Số dòng: 4898
    Số cột: 12
```

Kết quả cần được ghi trực tiếp theo kết quả chạy thực tế của Colab.

Không nên tự điền số liệu vào báo cáo nếu chưa chạy notebook cuối cùng.

3.2.7. Thống kê mô tả

```
df_spark.describe().show()
```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
|summary|fixed acidity|volatile acidity|citric acid|residual sugar|chlorides|free sulfur dioxide|total
+-----+-----+-----+-----+-----+-----+-----+-----+
|count|4890|4891|4896|4896|4896|4898|4898|
|mean|6.85553169734149|0.27825189122878863|0.33425040849672555|6.39324959150328|0.04577839052287585|35.30808493262556|138.
|stddev|0.8438075744270377|0.10081138482750696|0.12098510278902481|5.0722746628271365|0.02184993182318187|17.00713732523259|42.
|min|3.8|0.08|0.0|0.6|0.009|2.0|2.0|
|max|14.2|1.1|1.66|65.8|0.346|289.0|289.0|
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Thống kê mô tả giúp nhóm quan sát các đại lượng như giá trị trung bình, độ lệch chuẩn, giá trị nhỏ nhất và lớn nhất của các thuộc tính.

Qua bước này, nhóm có thể phát hiện những thuộc tính có phạm vi giá trị rất khác nhau, đồng thời nhận biết sơ bộ những giá trị bất thường.

3.2.8. Kiểm tra giá trị thiếu

```
null_col = null_counts.collect()[0]
for value in null_row:
    total_null = sum(
        value if value is not None else 0
    )

print("Tổng số giá trị thiếu:", total_null)
```

fixed	pH	sulphate
Tổng số giá trị thiếu: 30	7	

Việc kiểm tra và xử lý giá trị thiếu (NULL hoặc dữ liệu trống) là một bước mấu chốt và bắt buộc trong giai đoạn tiền xử lý dữ liệu, bởi vì nếu tồn tại các giá trị khuyết thiếu, các phép tính thống kê mô tả, quá trình biến đổi đặc trưng cũng như giai đoạn huấn luyện mô hình học máy sẽ dễ bị sai lệch, gây gián đoạn thuật toán hoặc làm giảm sút nghiêm trọng độ chính xác của mô hình dự đoán. Nhằm đảm bảo chất lượng đầu vào cho các bước tiếp theo, nhóm tiến hành rà soát toàn bộ cấu trúc DataFrame trên môi trường Spark, sau đó thực thi câu lệnh tổng hợp để tính toán tổng số lượng giá trị thiếu (NULL hoặc NaN) xuất hiện trên từng cột thuộc tính hóa học, từ đó đưa ra phương án xử lý phù hợp (như loại bỏ hoặc điền giá trị thay thế) trước khi đưa dữ liệu vào các bước phân tích sâu hơn.

3.2.9. Xử lý giá trị thiếu

```

▶ df_clean = df_spark.dropna()
print("Số dòng ban đầu:", df_spark.count())
print("Số dòng sau khi xử lý:", df_clean.count())
print("Số dòng bị loại:", df_spark.count() - df_clean.count())

... Số dòng ban đầu: 4898
Số dòng sau khi xử lý: 4870
Số dòng bị loại: 28

```

Sau khi xử lý, nhóm so sánh số lượng bản ghi trước và sau xử lý để xác định mức độ ảnh hưởng của bước này.

Nếu kết quả kiểm tra cho thấy dữ liệu không chứa giá trị thiếu thì có thể ghi:

Kết quả kiểm tra cho thấy bộ dữ liệu không phát sinh giá trị thiếu. Vì vậy, bước xử lý dropna() không làm thay đổi đáng kể số lượng bản ghi. Dữ liệu có thể tiếp tục được sử dụng cho các bước phân tích và xây dựng mô hình.

3.2.10. Kiểm tra dữ liệu bất hợp lệ

Một số thuộc tính có giới hạn ý nghĩa nhất định. Ví dụ, pH phải nằm trong khoảng hợp lý và các đại lượng như density hoặc alcohol không thể có giá trị bằng 0 hoặc âm.

```

invalid_count = df_clean.filter(
    (col("pH") <= 0) | (col("pH") > 14) |
    (col("density") <= 0) | (col("alcohol") <= 0)
).count()
print("Số dòng có giá trị bất hợp lệ:", invalid_count)

Số dòng có giá trị bất hợp lệ: 0

```

3.2.11. Xử lý dữ liệu bất hợp lệ

```

df_clean = df_clean.filter(
    (col("pH") > 0) & (col("pH") <= 14) &
    (col("density") > 0) & (col("alcohol") > 0)
)
print("Số dòng sau khi xử lý:", df_clean.count())

Số dòng sau khi xử lý: 4870

```

3.2.12. Phát hiện Outlier bằng IQR

Outlier là những giá trị nằm cách xa phần lớn dữ liệu. Nếu không được phân tích, các giá trị bất thường có thể ảnh hưởng đến thống kê và quá trình xây dựng mô hình.

Nhóm sử dụng phương pháp IQR:

$$IQR = Q3 - Q1$$

Trong đó Q1 là tứ phân vị thứ nhất và Q3 là tứ phân vị thứ ba.

Khoảng được xem là bình thường:

$$[Q1 - 1.5IQR, Q3 + 1.5IQR]$$

```
numeric_cols = [
    "fixed acidity", "volatile acidity", "citric acid",
    "residual sugar", "chlorides", "free sulfur dioxide",
    "total sulfur dioxide", "density", "pH", "sulphates", "alcohol"
]

for c in numeric_cols:
    q1, q3 = df_clean.approxQuantile(c, [0.25, 0.75], 0.01)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    count_outlier = df_clean.filter((col(c) < lower) | (col(c) > upper)).count()
    print(f"{c}: Q1={q1:.3f}, Q3={q3:.3f}, Outlier={count_outlier}")

... fixed acidity: Q1=6.300, Q3=7.300, Outlier=118
    volatile acidity: Q1=0.210, Q3=0.320, Outlier=182
    citric acid: Q1=0.270, Q3=0.380, Outlier=326
    residual sugar: Q1=1.700, Q3=9.700, Outlier=9
    chlorides: Q1=0.036, Q3=0.050, Outlier=206
    free sulfur dioxide: Q1=23.000, Q3=45.000, Outlier=57
    total sulfur dioxide: Q1=108.000, Q3=166.000, Outlier=20
    density: Q1=0.992, Q3=0.996, Outlier=5
    pH: Q1=3.080, Q3=3.280, Outlier=64
    sulphates: Q1=0.410, Q3=0.540, Outlier=183
    alcohol: Q1=9.500, Q3=11.333, Outlier=1
```

Sau đó nhóm đếm số lượng giá trị nằm ngoài khoảng cho từng thuộc tính.

3.2.13. Xử lý biến Quality

Nhóm chỉ sử dụng các mức Quality từ 4 đến 8:

Sau đó tạo biến label:

```
df_clean = df_clean.filter(col("quality").isin([4, 5, 6, 7, 8]))

df_clean = df_clean.withColumn(
    "label",
    when(col("quality") == 4, 0)
    .when(col("quality") == 5, 1)
    .when(col("quality") == 6, 2)
    .when(col("quality") == 7, 3)
    .when(col("quality") == 8, 4)
)

df_clean.groupBy("quality", "label").count().orderBy("quality").show()
```

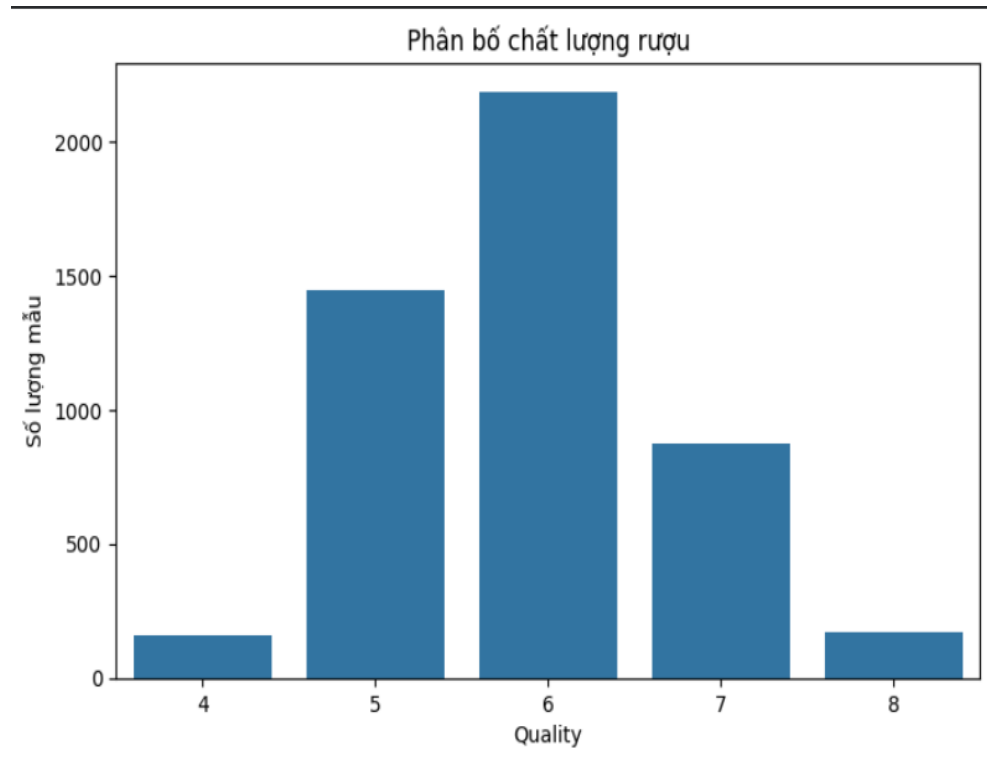
```
... +-----+-----+-----+
|quality|label|count|
+-----+-----+-----+
|      4|    0|   162|
|      5|    1|  1448|
|      6|    2|  2186|
|      7|    3|   875|
|      8|    4|   174|
+-----+-----+-----+
```

Việc chuyển đổi biến mục tiêu (quality) sang dạng nhãn số nguyên tuần tự là một bước bắt buộc và vô cùng quan trọng, giúp các giá trị này tương thích hoàn toàn với định dạng chuẩn mực mà các thuật toán phân loại (Classification) trong thư viện Spark MLlib yêu cầu. Cụ thể, các mô hình học máy phân loại của Spark thường đòi hỏi nhãn đầu ra phải bắt đầu từ số 0 và tăng dần liên tục theo số lượng lớp, thay vì giữ nguyên các giá trị rời rạc hoặc phân tán ban đầu. Thông qua quá trình ánh xạ này, hệ thống có thể hiểu và xử lý chính xác cấu trúc bài toán đa lớp, đồng thời tránh phát sinh các lỗi ngoại lệ hoặc sai sót trong quá trình huấn luyện và đánh giá mô hình trên nền tảng phân tán.

3.2.14. Phân tích khám phá dữ liệu

3.2.14.1. Phân bố Quality

Biểu đồ phân bố Quality được sử dụng để xác định số lượng mẫu ở từng mức chất lượng.



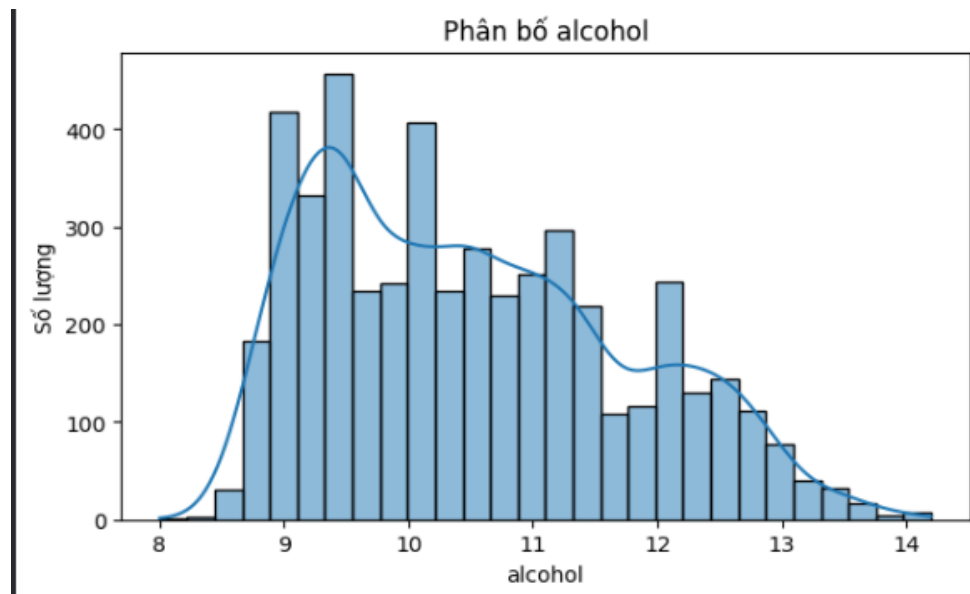
Hình 3.3: Phân bố chất lượng rượu

Nhận xét:

Dựa trên biểu đồ, nhóm có thể xác định mức Quality nào xuất hiện nhiều nhất và mức nào có số lượng mẫu ít hơn. Nếu các lớp không cân bằng, điều này cần được lưu ý khi đánh giá mô hình.

3.2.14.2. Phân bố Alcohol

Biểu đồ histogram được sử dụng để quan sát phạm vi và hình dạng phân bố của nồng độ alcohol.

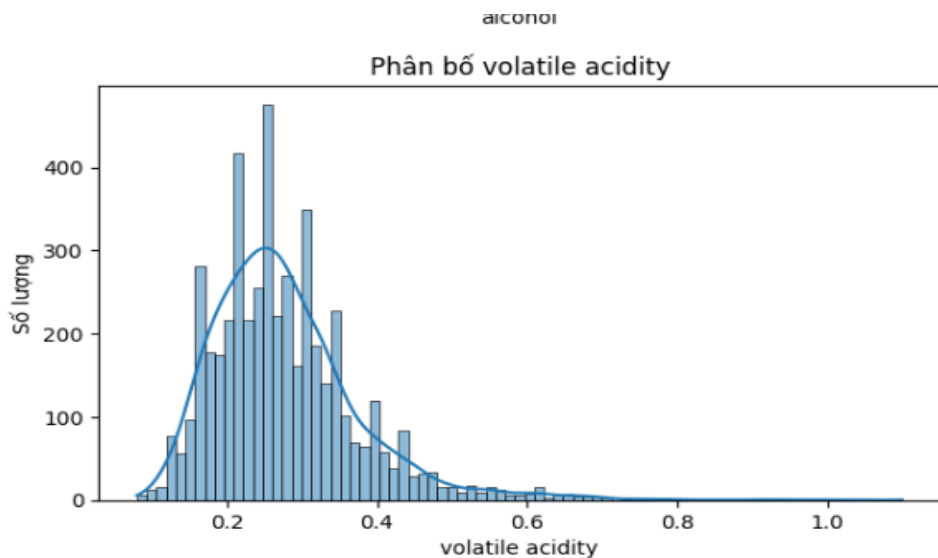


Hình 3.4: Phân bố Alcohol

Nhận xét:

Biểu đồ cho phép quan sát vùng giá trị tập trung của nồng độ alcohol, đồng thời nhận biết những giá trị nằm xa phần lớn dữ liệu.

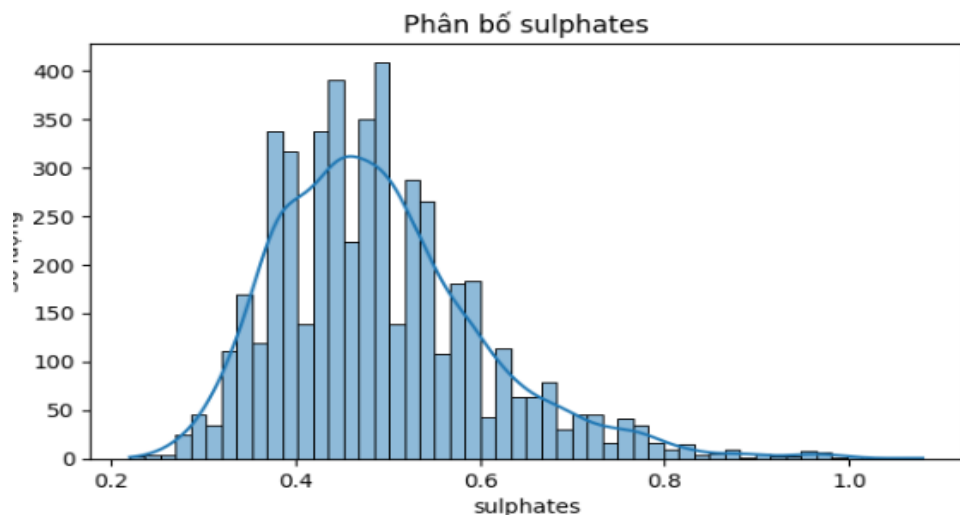
3.2.14.3. Phân bố Volatile Acidity



Hình 3.5: Phân bố Volatile Acidity

Phân tích thuộc tính này giúp nhóm quan sát mức độ phân tán và hình dạng phân phối của độ acid dễ bay hơi.

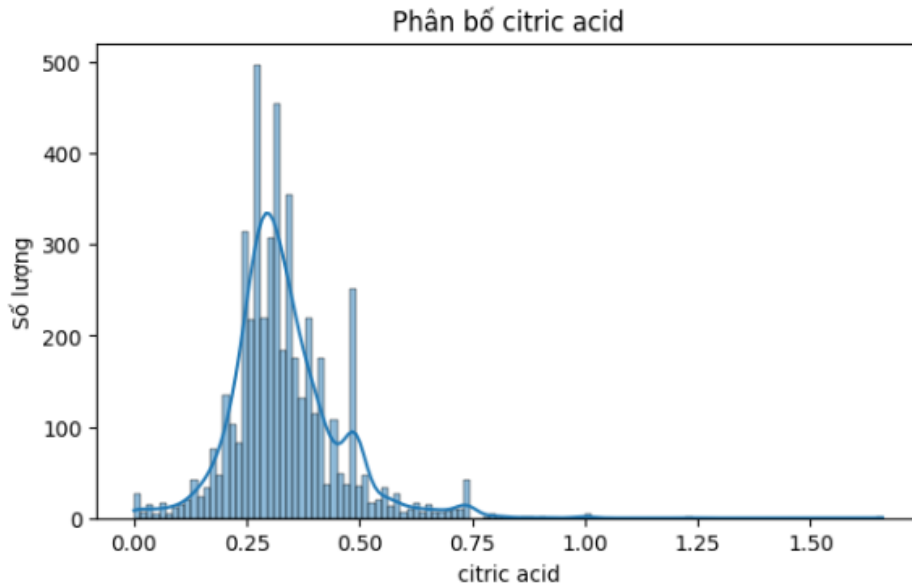
3.2.14.4. Phân bố Sulphates



Hình 3.6: Phân bố Sulphates

Biểu đồ giúp quan sát phạm vi giá trị của sulphates và những vùng dữ liệu tập trung nhiều mẫu.

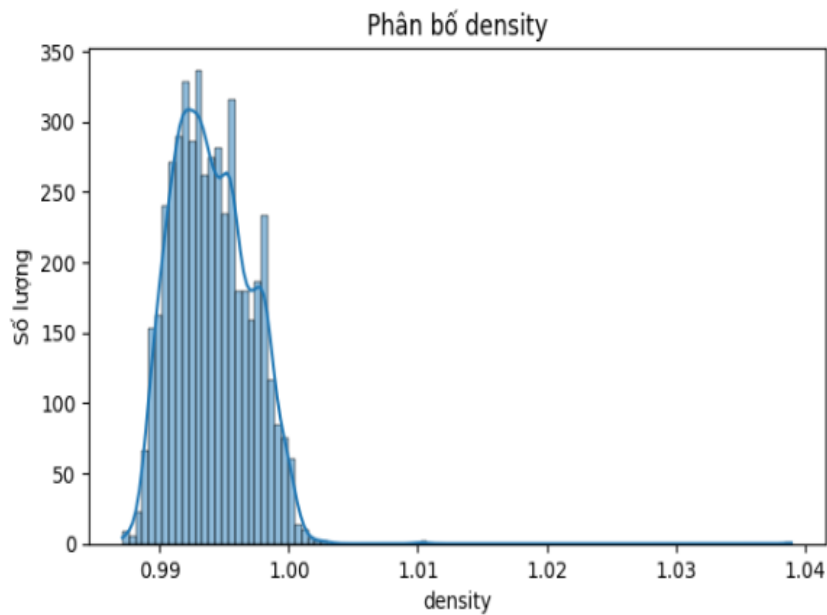
3.2.14.5. Phân bố Citric Acid



Hình 3.7: Phân bố Citric Acid

Qua biểu đồ, nhóm có thể đánh giá mức độ phân tán của acid citric trong các mẫu rượu.

3.2.14.6. Phân bố Density



Hình 3.8: Phân bố Density

Density là một trong những thuộc tính vật lý của rượu. Việc quan sát phân bố giúp kiểm tra phạm vi dữ liệu và phát hiện những giá trị bất thường.

3.2.15. Phân tích tương quan

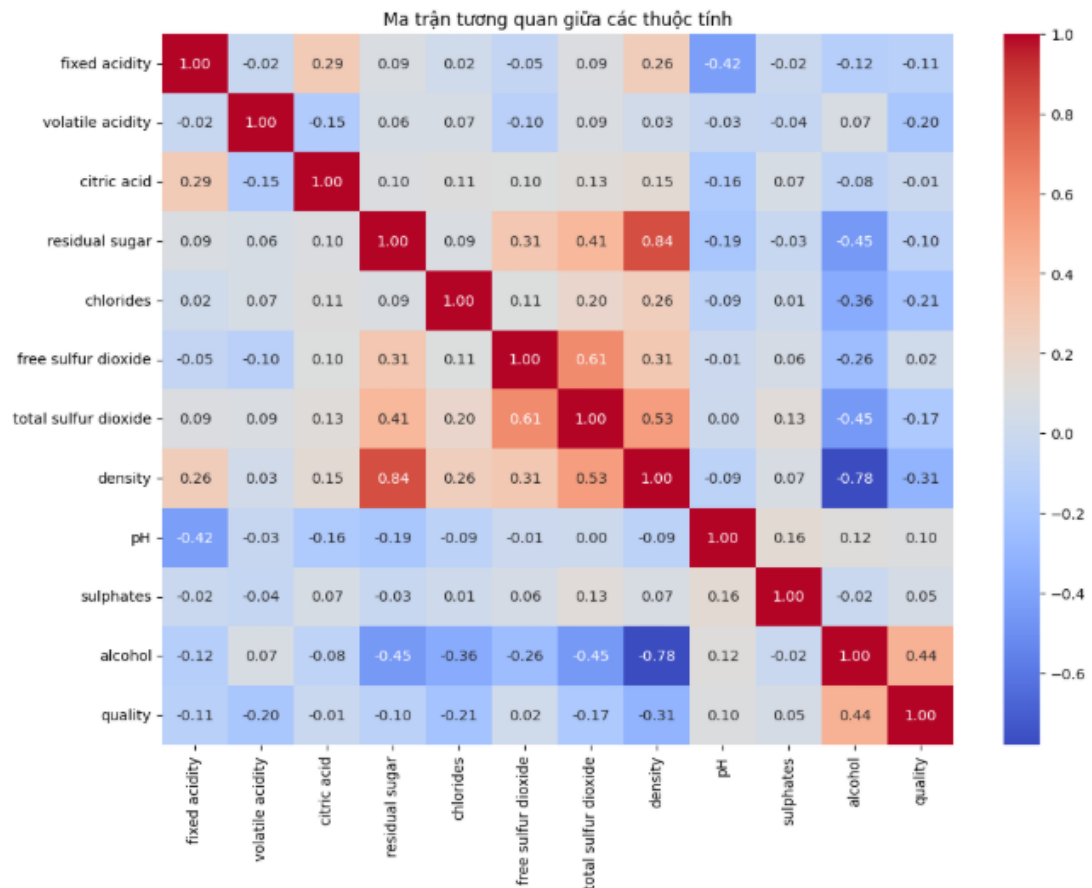
3.2.15.1. Ma trận tương quan

```

corr = df[numeric_cols + ["quality"]].corr()
plt.figure(figsize=(12, 9))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Ma trận tương quan giữa các thuộc tính")
plt.show()

```

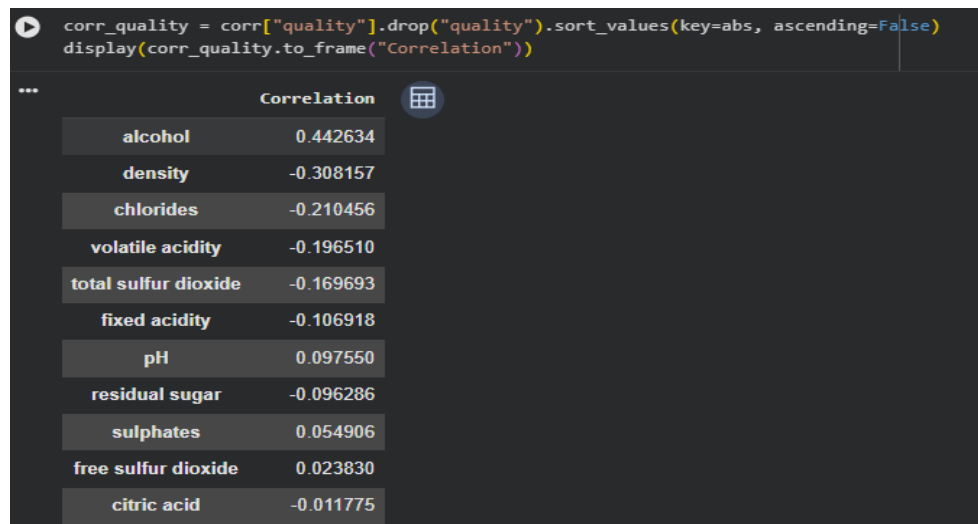
Sau đó trực quan hóa bằng Heatmap.



Hình 3.9: Ma trận tương quan giữa các thuộc tính

Ma trận tương quan đóng vai trò là một công cụ phân tích khám phá dữ liệu quan trọng, giúp nhóm xác định rõ ràng mức độ liên hệ tuyến tính giữa các thuộc tính hóa học. Cụ thể, các giá trị tương quan dương cho thấy hai biến có xu hướng biến động cùng chiều, trong khi hệ số tương quan âm thể hiện xu hướng ngược chiều rõ rệt. Qua đó, phân tích này cung cấp cơ sở vững chắc để lựa chọn các đặc trưng tối ưu và hạn chế hiện tượng đa cộng tuyến trước khi đưa dữ liệu vào mô hình.

3.2.15.2. Tương quan giữa thuộc tính và Quality



Hình 3.10: Mức độ tương quan giữa các thuộc tính và Quality

Nhận xét:

Thuộc tính có tương quan dương cao nhất: Thuộc tính alcohol (nồng độ cồn) đạt mức tương quan dương cao nhất với hệ số khoảng 0.443, cho thấy đây là yếu tố tác động mạnh mẽ và thuận chiều nhất đến chất lượng rượu; các mẫu rượu có nồng độ cồn cao hơn thường có xu hướng đạt điểm chất lượng tốt hơn. Các thuộc tính khác như pH hay sulphates cũng có tương quan dương nhưng ở mức độ thấp hơn.

Thuộc tính có tương quan âm đáng kể: Ngược lại, thuộc tính density (mật độ) và chlorides (nồng độ clorua) thể hiện hệ số tương quan âm lần lượt là -0.308 và -0.210, thể hiện xu hướng nghịch chiều (mật độ hoặc hàm lượng muối cao hơn thường gắn liền với mức đánh giá chất lượng thấp hơn). Các yếu tố như volatile acidity, total sulfur dioxide hay fixed acidity cũng mang giá trị âm, phản ánh tác động ngược chiều đến điểm chất lượng.

Thuộc tính ít ảnh hưởng: Một số đặc trưng như free sulfur dioxide hay citric acid có giá trị hệ số tương quan rất sát ngưỡng 0 (ví dụ citric acid xấp xỉ -0.012), cho thấy mối liên hệ tuyến tính giữa các thuộc tính này với chất lượng rượu là không đáng kể trong tập dữ liệu khảo sát.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Chuẩn bị đặc trưng

Các thuộc tính được đưa vào mô hình:

```
feature_cols = [
    "fixed acidity",
    "volatile acidity",
    "citric acid",
    "residual sugar",
    "chlorides",
    "free sulfur dioxide",
    "total sulfur dioxide",
    "density",
    "pH",
    "sulphates",
    "alcohol"
]
```

4.2. Vector hóa dữ liệu

Sau bước này, 11 thuộc tính hóa học được kết hợp thành một vector đặc trưng.

4.3. Chuẩn hóa

```
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures",
    withStd=True,
    withMean=True
)

scaler_model = scaler.fit(df_vector)
df_scaled = scaler_model.transform(df_vector)

df_scaled.select("label", "features", "scaledFeatures").show(5, truncate=False)
```

label	features	scaledFeatures
2	[7.0, 0.27, 0.36, 20.7, 0.045, 45.0, 170.0, 1.001, 3.0, 0.45, 8.8]	[0.17753670710983124, -0.07797368113039121, 0.21358365882404964, 2.8204270654541204, -0.03481577586404292, 0.5930958107377061, 0.7564134547
2	[6.3, 0.3, 0.34, 1.6, 0.049, 14.0, 132.0, 0.994, 3.3, 0.49, 9.5]	[-0.6591418757610269, 0.22106258674037366, 0.048410023229076726, -0.9458470118998578, 0.149586994358748, -1.2916372904333797, -0.1483312506
2	[8.1, 0.28, 0.4, 6.9, 0.05, 30.0, 97.0, 0.9951, 3.26, 0.44, 10.1]	[1.4923173373354648, 0.021705074826530595, 0.5439309300139964, 0.09924474778475391, 0.19568768691444574, -0.3188718188612064, -0.9816487424
2	[7.2, 0.23, 0.32, 8.5, 0.058, 47.0, 186.0, 0.9956, 3.19, 0.4, 9.9]	[0.41658773078721945, -0.4766887049580782, -0.11676361236589665, 0.4147441469348253, 0.5644932273600273, 0.7146914946842279, 1.137358593823
2	[7.2, 0.23, 0.32, 8.5, 0.058, 47.0, 186.0, 0.9956, 3.19, 0.4, 9.9]	[0.41658773078721945, -0.4766887049580782, -0.11676361236589665, 0.4147441469348253, 0.5644932273600273, 0.7146914946842279, 1.137358593823

only showing top 5 rows

4.4. Chia dữ liệu Train/Test

```
train_df, test_df = df_scaled.randomSplit([0.8, 0.2], seed=42)
print("Số mẫu Train:", train_df.count())
print("Số mẫu Test :", test_df.count())
```

```
Số mẫu Train: 3917
Số mẫu Test : 928
```

Nhóm tiến hành phân chia tập dữ liệu theo tỷ lệ tiêu chuẩn 80% cho quá trình huấn luyện (training set) mô hình và 20% còn lại dùng để kiểm tra, đánh giá (testing set). Bên cạnh đó, việc cố định giá trị seed = 42 đóng vai trò rất quan trọng, giúp đảm bảo quá trình

xáo trộn và chia tách dữ liệu luôn diễn ra theo đúng một trật tự cố định, từ đó giúp kết quả có thể tái lập hoàn toàn mỗi khi chạy lại notebook trên môi trường Spark.

4.5. Mô hình Logistic Regression

Logistic Regression sử dụng scaledFeatures làm biến đầu vào vì đây là tập dữ liệu đã được chuẩn hóa thông qua StandardScaler, giúp các đặc trưng đưa về cùng một mặt bằng thang đo chung. Sau khi quá trình huấn luyện hoàn tất, các kết quả dự đoán từ mô hình sẽ được thu thập và đối chiếu với nhãn thực tế để tiến hành tính toán chi tiết các chỉ số đánh giá hiệu suất.

4.6. Mô hình Decision Tree

```
dt = DecisionTreeClassifier(featuresCol="features", labelCol="label", maxDepth=5, seed=42)
dt_model = dt.fit(train_df)
dt_pred = dt_model.transform(test_df)
dt_pred.select("label", "prediction").show(10)
```

label	prediction
3	3.0
1	1.0
2	2.0
3	3.0
1	1.0
4	3.0
3	2.0
2	3.0
2	2.0
2	3.0

only showing top 10 rows

Mô hình Decision Tree (Cây quyết định) sử dụng toàn bộ tập hợp các đặc trưng đầu vào để tiến hành phân tách dữ liệu một cách đệ quy và xây dựng nên cấu trúc cây phân loại trực quan. Thông qua các quy tắc chia nhánh dựa trên việc lựa chọn thuộc tính tối ưu và các ngưỡng giá trị tương ứng của từng thuộc tính hóa học, mô hình có khả năng tự động học hỏi các quy luật phức tạp để phân loại các mẫu dữ liệu vào các nhãn chất lượng rượu tương ứng, đồng thời mang lại ưu thế lớn về khả năng giải thích logic của quyết định. Hơn thế nữa, cấu trúc phân cấp này giúp thuật toán dễ dàng nắm bắt các mối quan hệ phi tuyến tính tiềm ẩn giữa các thông số hóa học mà các mô hình tuyến tính có thể bỏ sót, từ đó cung cấp một góc nhìn trực quan và minh bạch về cách thức hệ thống đưa ra các phán đoán phân loại cuối cùng cho từng mẫu sản phẩm.

4.7. Mô hình Random Forest

```
rf = RandomForestClassifier(  
    featuresCol="features",  
    labelCol="label",  
    numTrees=100,  
    maxDepth=8,  
    seed=42  
)  
rf_model = rf.fit(train_df)  
rf_pred = rf_model.transform(test_df)  
rf_pred.select("label", "prediction").show(10)
```

```
+-----+-----+  
|label|prediction|  
+-----+-----+  
|    3|      3.0|  
|    1|      1.0|  
|    2|      2.0|  
|    3|      3.0|  
|    1|      2.0|  
|    4|      3.0|  
|    3|      2.0|  
|    2|      2.0|  
|    2|      2.0|  
|    2|      3.0|  
+-----+-----+  
only showing top 10 rows
```

Random Forest được lựa chọn vì có khả năng kết hợp nhiều Decision Tree và xử lý tốt các mối quan hệ phức tạp giữa các thuộc tính.

4.8. Hàm đánh giá mô hình

Nhóm sử dụng Accuracy, Precision, Recall và F1-score:

```
def evaluate_model(predictions, model_name):  
    evaluator = MulticlassClassificationEvaluator(  
        labelCol="label",  
        predictionCol="prediction"  
    )  
    accuracy = evaluator.evaluate(predictions, {evaluator.metricName: "accuracy"})  
    precision = evaluator.evaluate(predictions, {evaluator.metricName: "weightedPrecision"})  
    recall = evaluator.evaluate(predictions, {evaluator.metricName: "weightedRecall"})  
    f1 = evaluator.evaluate(predictions, {evaluator.metricName: "f1"})  
    return {  
        "Model": model_name,  
        "Accuracy": accuracy,  
        "Precision": precision,  
        "Recall": recall,  
        "F1": f1  
    }
```

4.9. So sánh các mô hình

Nhận xét:

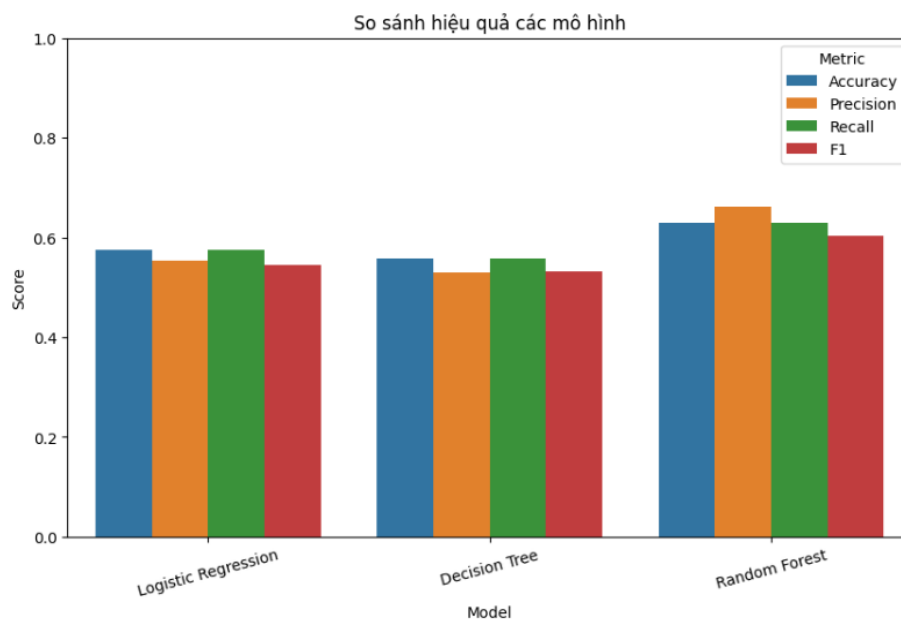
Kết quả của ba mô hình cần được phân tích dựa trên nhiều chỉ số thay vì chỉ dựa vào Accuracy. Trong trường hợp dữ liệu có sự chênh lệch giữa các lớp Quality, F1-score có thể cung cấp góc nhìn cân bằng hơn về khả năng phân loại của mô hình.

Mô hình có F1-score cao nhất được xem xét là mô hình phù hợp nhất với bài toán.

4.10. Biểu đồ so sánh

Nhóm sử dụng biểu đồ để so sánh:

- Accuracy.
- Precision.
- Recall.
- F1-score.



Hình 4.1: So sánh hiệu quả các mô hình

Việc trực quan hóa dữ liệu thông qua biểu đồ đóng vai trò vô cùng quan trọng, giúp các thành viên trong nhóm dễ dàng so sánh trực quan hiệu suất giữa các mô hình học máy với nhau một cách nhanh chóng và trực quan nhất. Nhờ vào sự thể hiện rõ ràng của các cột hoặc đường biểu diễn, người phân tích có thể nhanh chóng nhận diện và xác định chính xác mô hình nào đang đạt hiệu năng tối ưu, từ đó có cơ sở vững chắc để đưa ra lựa chọn thuật toán phù hợp nhất cho toàn bộ bài toán đề án.

4.11. Confusion Matrix

Đối với Random Forest:

```
rf_result_pd = rf_pred.select("label", "prediction").toPandas()
cm = confusion_matrix(rf_result_pd["label"], rf_result_pd["prediction"])

plt.figure(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

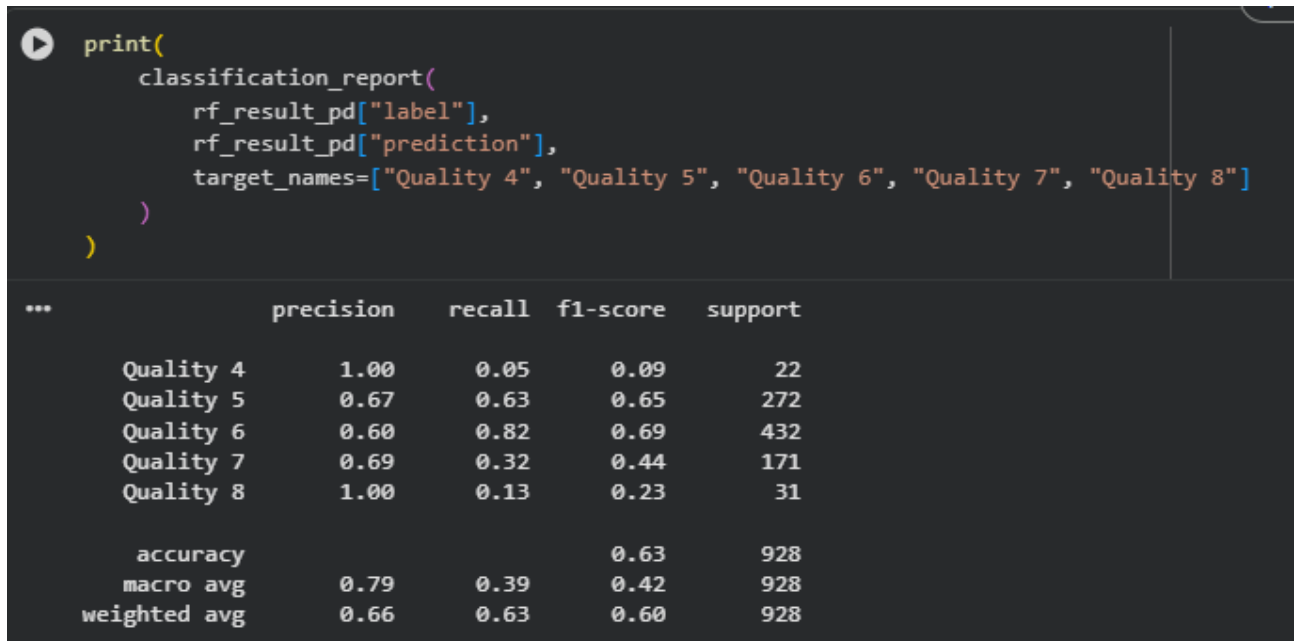


Hình 4.2: Confusion Matrix – Random Forest

Confusion Matrix cho biết số lượng mẫu thực tế thuộc từng lớp và số lượng mẫu mà mô hình dự đoán sang từng lớp. Từ đó có thể xác định mô hình thường nhầm lẫn giữa Quality nào với Quality nào để đánh giá chi tiết sai số phân loại và tìm ra nguyên nhân cụ thể dẫn đến sự sai lệch trong quá trình dự đoán nhằm cải thiện độ chính xác của hệ thống học máy.

4.12. Classification Report

Classification Report cung cấp Precision, Recall và F1-score cho từng lớp Quality.



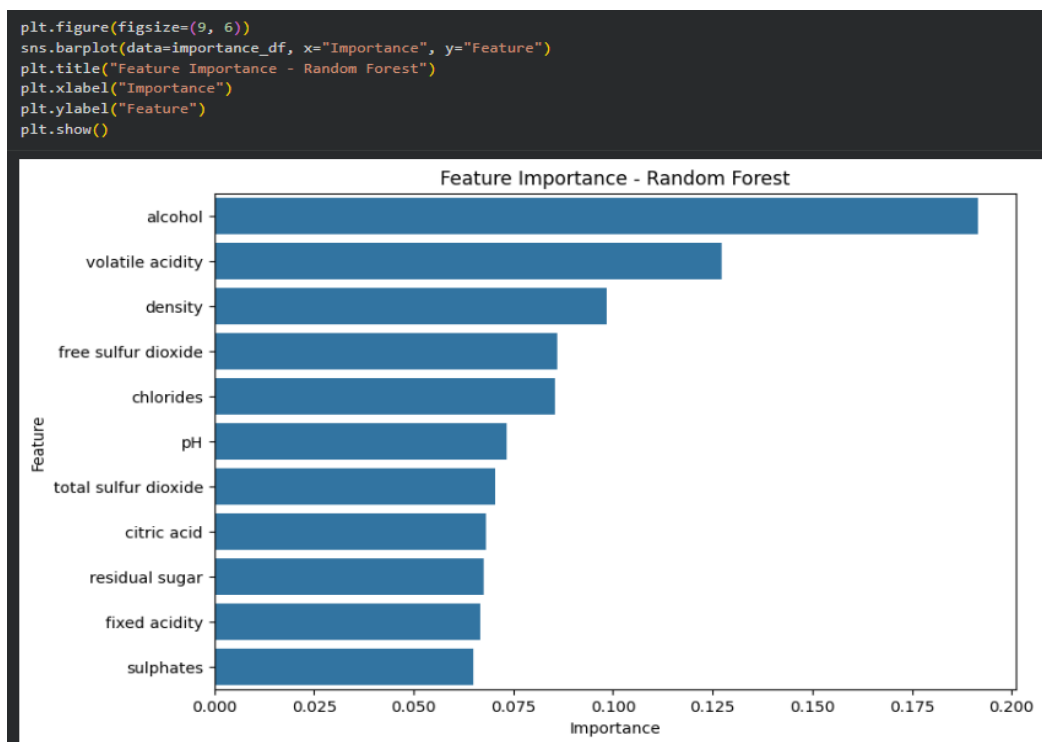
Hình 4.3: Classification Report

Phần này giúp đánh giá chi tiết hơn so với chỉ sử dụng Accuracy tổng thể.

4.13. Feature Importance

Random Forest cung cấp thông tin về mức độ quan trọng của từng đặc trưng.

Sau đó nhóm xây dựng bảng:



Nhận xét:

Feature Importance giúp xác định những thuộc tính đóng góp nhiều hơn vào quá trình phân loại của Random Forest. Tuy nhiên, mức độ quan trọng của một biến trong mô hình không đồng nghĩa với việc biến đó là nguyên nhân trực tiếp tạo ra chất lượng rượu.

4.14. Cross Validation

Để cải thiện cấu hình Random Forest, nhóm sử dụng Cross Validation với các giá trị khác nhau của numTrees và maxDepth.

```
param_grid = ParamGridBuilder().addGrid(rf.numTrees, [50, 100]).addGrid(rf.maxDepth, [5, 8]).build()

evaluator_cv = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="f1"
)

cv = CrossValidator(
    estimator=rf,
    estimatorParamMaps=param_grid,
    evaluator=evaluator_cv,
    numFolds=3,
    seed=42
)

cv_model = cv.fit(train_df)
cv_pred = cv_model.transform(test_df)
cv_f1 = evaluator_cv.evaluate(cv_pred)

print("F1-score sau Cross Validation:", round(cv_f1, 4))
```

F1-score sau Cross Validation: 0.6105

Cross Validation giúp hạn chế việc lựa chọn một cấu hình mô hình chỉ dựa trên một lần chia dữ liệu.

4.15. Dự đoán mẫu rượu mới

Nhóm tạo một mẫu dữ liệu mới gồm 11 thông số hóa học:

Sau đó thực hiện VectorAssembler và đưa mẫu vào mô hình.

Kết quả dự đoán ban đầu là Label. Vì vậy nhóm chuyển ngược Label về Quality:

```
sample = spark.createDataFrame(
    [(7.4, 0.70, 0.00, 1.9, 0.076, 11.0, 34.0, 0.9978, 3.51, 0.56, 9.4)],
    feature_cols
)
sample.show()
```

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
7.4	0.7	0.0	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4

Hình 4.4: Kết quả dự đoán mẫu rượu mới

Đây là bước minh họa khả năng ứng dụng mô hình sau khi đã được huấn luyện.

CHƯƠNG 5: KẾT LUẬN

5.1. Kết luận

Sau quá trình thực nghiệm, nhóm đã xây dựng được một quy trình tương đối hoàn chỉnh để sử dụng Apache Spark trong bài toán dự đoán chất lượng rượu.

Quy trình bắt đầu từ việc đọc dữ liệu bằng Spark, kiểm tra cấu trúc và thống kê mô tả, sau đó thực hiện kiểm tra dữ liệu thiếu và dữ liệu bất hợp lệ. Việc phân tích Outlier bằng phương pháp IQR giúp nhóm có thêm thông tin về chất lượng dữ liệu trước khi đưa vào mô hình.

Tiếp theo, nhóm tiến hành EDA để quan sát sự phân bố của Quality và các thuộc tính hóa học. Ma trận tương quan được sử dụng để tìm hiểu mối liên hệ giữa các biến và chất lượng rượu.

Sau quá trình phân tích, các đặc trưng được chuyển thành vector bằng VectorAssembler và chuẩn hóa bằng StandardScaler. Dữ liệu được chia thành tập Train và Test với tỷ lệ 80/20.

Ba mô hình Machine Learning được xây dựng bằng Spark MLlib gồm Logistic Regression, Decision Tree và Random Forest. Các mô hình được đánh giá thông qua Accuracy, Precision, Recall và F1-score.

Random Forest còn được sử dụng để phân tích Feature Importance, qua đó giúp nhóm xác định những thuộc tính có đóng góp đáng kể vào quá trình dự đoán.

Cuối cùng, nhóm sử dụng Cross Validation để đánh giá các cấu hình Random Forest và thực hiện dự đoán chất lượng cho một mẫu rượu mới.

5.2. Ưu điểm của đề tài

Một ưu điểm của đề tài là toàn bộ quá trình xử lý dữ liệu và Machine Learning được xây dựng dựa trên hệ sinh thái Apache Spark. Điều này giúp nhóm có cơ hội áp dụng kiến thức về xử lý dữ liệu phân tán vào một bài toán cụ thể.

Bên cạnh đó, việc sử dụng nhiều mô hình giúp nhóm có cơ sở so sánh hiệu quả thay vì chỉ xây dựng một mô hình duy nhất.

Đề tài cũng không chỉ dừng lại ở việc đánh giá Accuracy mà sử dụng thêm Precision, Recall, F1-score và Confusion Matrix để phân tích mô hình toàn diện hơn.

Việc bổ sung Feature Importance và Cross Validation giúp phần thực nghiệm có

chiều sâu hơn so với một quy trình chỉ huấn luyện và dự đoán.

5.3. Hạn chế của đề tài

Mặc dù sử dụng Apache Spark, bộ dữ liệu Wine Quality được sử dụng trong đề tài không quá lớn. Vì vậy, lợi thế xử lý dữ liệu phân tán của Spark chưa thể hiện rõ như khi xử lý những tập dữ liệu có kích thước rất lớn.

Môi trường thực nghiệm được triển khai trên Google Colab nên nhóm chưa xây dựng một Spark Cluster thực tế gồm nhiều Worker Node. Do đó, đề tài chủ yếu minh họa cách sử dụng PySpark và Spark MLlib thay vì đánh giá hiệu năng của một hệ thống Spark phân tán thực tế.

Ngoài ra, chất lượng rượu là một đặc điểm có thể chịu ảnh hưởng của nhiều yếu tố khác nhau. Bộ dữ liệu hiện tại chủ yếu mô tả các đặc trưng hóa học nên chưa bao quát tất cả yếu tố có thể ảnh hưởng đến chất lượng sản phẩm.

Các mô hình được sử dụng trong đề tài cũng mới tập trung vào một số thuật toán cơ bản của Spark MLlib. Việc tối ưu mô hình vẫn có thể được mở rộng thêm trong các nghiên cứu tiếp theo.

5.4. Hướng phát triển

Trong tương lai, đề tài có thể được phát triển theo nhiều hướng.

Thứ nhất, nhóm có thể mở rộng bộ dữ liệu bằng cách thu thập thêm nhiều mẫu rượu từ các nguồn khác nhau. Dataset lớn hơn sẽ phù hợp hơn để đánh giá khả năng xử lý dữ liệu lớn của Apache Spark.

Thứ hai, nhóm có thể triển khai Spark trên một Cluster thực tế gồm nhiều Worker Node để đánh giá thời gian xử lý, khả năng mở rộng và hiệu suất khi số lượng dữ liệu tăng lên.

Thứ ba, có thể thử nghiệm thêm các thuật toán Machine Learning khác trong Spark MLlib và thực hiện tối ưu Hyperparameter sâu hơn.

Thứ tư, nhóm có thể xây dựng một giao diện cho phép người dùng nhập các thông số hóa học của một mẫu rượu. Dữ liệu sau khi nhập sẽ được đưa vào mô hình đã huấn luyện và trả về chất lượng rượu dự đoán.

Cuối cùng, mô hình có thể được triển khai thành một API hoặc ứng dụng web, giúp người dùng có thể sử dụng hệ thống dự đoán trực tiếp thay vì chạy notebook.

TÀI LIỆU THAM KHẢO

- [1] Karau, H., Konwinski, A., Wendell, P., & Zaharia, M. (2015). Learning Spark: Lightning-Fast Big Data Analysis. O'Reilly Media.
- [2] Chambers, B., & Zaharia, M. (2018). Spark: The Definitive Guide: Big Data Processing Made Simple. O'Reilly Media.
- [3] White, T. (2015). Hadoop: The Definitive Guide (4th Edition). O'Reilly Media.
<https://www.oreilly.com/library/view/hadoop-the-definitive/9781491901633/>
- [4] Databricks. (2024). What is Apache Spark? <https://www.databricks.com/glossary/what-is-apache-spark>
- [5] Phan, T. A., & Nguyễn, V. H. (2020). Giáo trình Công nghệ Big Data và ứng dụng phân tích dữ liệu. Nhà xuất bản Đại học Quốc gia Hà Nội.

PHỤ LỤC

Dataset(csv): <https://kaggle.com/datasets/ruthgn/wine-quality-data-set-red-white-wine/data>

Github: [https://github.com/hoangnguyenhoang2005
create/Bigdata_HoangNguyenHoang_2311553873](https://github.com/hoangnguyenhoang2005/create/Bigdata_HoangNguyenHoang_2311553873)